

Lab 2D: Ship It

Duration: 30-45 min | **Difficulty:** Intermediate

After: Git Workflow & Best Practices slides

Continues from: Lab 2C (your Business Dashboard should have separate route files and a task form)

© 2026 AIA Copilot — Instructor-Led Training Material

Goal

Polish the dashboard into something professional, then use Git workflows in natural language. Start rough, iterate to great — this is how professionals use Claude Code every day.

Steps

1. Add Data Visualization

Add a simple bar chart to the dashboard showing task counts by status (pending, in-progress, completed). Use vanilla JavaScript and CSS — no chart libraries. Make it look good with our dark theme colors.

2. Verify in Browser

Open `http://localhost:3100` and look at the chart.

3. Critique and Refine

Look at the result, then tell Claude what to improve:

The chart works but [describe what you'd change — colors, spacing, labels, etc.]
Make it better.

***This is the key skill.** You don't need to know how to fix it. You just need to know what "better" looks like and describe it. Claude handles the implementation.*

Try 2-3 rounds of refinement. Each time, describe what's not right and let Claude fix it.

4. Add Search and Filter

```
Add a filter bar above the tasks list:
- Dropdown to filter by status (All, Pending, In Progress, Completed)
- Dropdown to filter by priority (All, High, Medium, Low)
- Filters should update the list immediately (no page reload)
```

Test the filters in your browser.

5. Production Hardening

Ask Claude an architectural question:

```
What would you change about this codebase before deploying to production?
```

Review the suggestions, then:

```
Implement the top 3 most important production improvements you just suggested.
```

***You just used Claude as an architectural partner,** not just a code generator. The "what would you change?" pattern is incredibly powerful — Claude knows the difference between demo code and production code.*

6. Commit Your Polish Work

```
Commit all improvements with message "feat: add chart, filters, and production
hardening"
```

Feature Branch Workflow

7. Create a Feature Branch

```
Create a new git branch called "feature/export" and switch to it
```

8. Build the Export Feature

```
Add a CSV export feature:  
1. New endpoint: GET /api/export/metrics?format=csv returns metrics as downloadable CSV  
2. New endpoint: GET /api/export/tasks?format=csv returns tasks as downloadable CSV  
3. Add "Export CSV" buttons to the dashboard UI for both metrics and tasks  
4. Buttons should trigger file downloads
```

9. Test the Export

Ask Claude to verify:

```
Test both CSV export endpoints and show me the output
```

Also test the buttons in your browser — they should download .csv files.

10. Commit and Merge

```
Commit the export feature, switch back to main, and merge the feature branch
```

11. Review Your Git History

```
Show me the git log with a nice one-line format
```

Expected output: Clean conventional commits telling the story of your project:

```
abc1234 feat: add CSV export for metrics and tasks  
def5678 feat: add chart, filters, and production hardening  
ghi9012 feat: add task creation form to frontend  
jkl3456 refactor: extract routes into separate files  
mno7890 feat: add task statistics endpoint  
pqr1234 feat: initial business dashboard with API and UI
```

Optional: Go Deeper

The "interview me" pattern:

```
I want to add user authentication to this dashboard. Before writing any code,
interview me: ask me 5 questions about my requirements so you build exactly
what I need.
```

Answer Claude's questions, then let it build the auth system. Notice how much better the result is when Claude asks first.

Code review your own project:

```
Review the entire codebase against our CLAUDE.md conventions.
Are there any violations? List them and fix the top 3.
```

UI polish round:

```
Look at @public/index.html – the dashboard needs these improvements:
1. Add a last-updated timestamp that refreshes every 30 seconds
2. Metrics cards should show a colored indicator: green for positive trends, red for is
   sues
3. The task list should show how long ago each task was created (e.g., "2 hours ago")
4. Add subtle hover effects on cards and task items
```

Add a completely new feature of YOUR choice:

Think about what would make this dashboard useful for YOUR business. Ask Claude to build it. Examples: - Email notifications when a high-priority task is created - A notes/comments field on each task - Metric history charts showing trends over time - User roles (admin vs viewer) - Dark/light theme toggle

❑ Lab 2D Final Checkpoint

Your completed Business Dashboard includes: - [] REST API with 10+ endpoints - [] SQLite database (zero configuration) - [] Professional dark-themed web UI - [] Input validation with proper error responses - [] Bar chart visualization - [] Task filters (status + priority) - [] CSV export for metrics and tasks - [] Add-task form on the frontend - [] Production hardening (env vars, security

headers, logging) - [] CLAUDE.md with team conventions - [] Clean git history with conventional commits - [] Feature branch workflow

What You Built vs. What You Typed

Count your prompts. You probably typed **fewer than 25 messages** to Claude Code across all four labs and got:

- ~500 lines of backend code
- ~200 lines of frontend code
- A working SQLite database with sample data
- A professional UI with charts, filters, and forms
- Production-quality error handling and security
- Clean git history with 6+ conventional commits

That's the leverage. You described what you wanted. Claude Code built it.

Key Takeaways

Lab	Concept	Key Lesson
2A	The Agentic Loop	You provide intent, Claude provides implementation
2B	CLAUDE.md	One file makes conventions automatic — policy, not documentation
2C	@ Mentions + Recovery	Precision saves tokens; recovery prompting rebuilds what's broken
2D	Iterate + Git	Start rough, polish progressively; Git in natural language

□ Continues in Day 3

Tomorrow, you'll take this exact dashboard and add: - Custom slash commands for common operations - Security hooks (secret detection, audit logging) - A report-generation skill - MCP integration for live data - Deployment with subagents working in parallel

Don't delete this project. You'll need it tomorrow.

Troubleshooting

Issue	Fix
Port 3100 in use	Tell Claude: "The port is in use, switch to 3200"
<code>better-sqlite3</code> won't install	Tell Claude: "better-sqlite3 failed to install, fix it"
Database locked	Tell Claude: "The database is locked, stop any running servers and restart"
Claude seems confused	Run <code>/clear</code> and re-state what you want with @ file references
Git conflicts	Tell Claude: "Resolve any git conflicts and show me the result"
Node version too old	Run <code>node -v</code> — must be 18+. Install latest from nodejs.org
Claude Code not found	See Pre-Flight section in Lab 2A for installation commands
Slow responses	Try <code>/model sonnet</code> for faster (but slightly less capable) responses
Context too large	Run <code>/compact</code> to compress conversation history
Claude not following conventions	Verify <code>CLAUDE.md</code> is in project root. Try <code>/clear</code> so Claude re-reads it
<code>tmpclaude-*</code> files in project	Temp files from Claude's editing process. Safe to delete
